

Exercise Session 6: Transformers, Diffusion Models, and Foundation Models

Automatic Image Analysis

Exercise Session 6

Transformers, Diffusion Models, and Foundation Models including DINOv2 and Stable Diffusion.

Computer Vision and Remote
Sensing Group, TU Berlin
Summer Semester 2026

- How does attention compute contextual representations?
- How do diffusion models learn to generate images?
- What makes a model foundational?

Roadmap

Part I: Transformers

- Motivation and embeddings
- Scaled dot-product attention
- Multi-head attention
- Transformer encoder block
- Vision Transformer (ViT)

Part II: Diffusion Models

- Forward noising process
- Reverse denoising process
- DDPM objective
- U-Net backbone
- Latent diffusion (LDM)

Part III: Applications

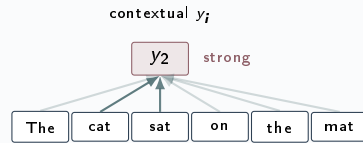
- Foundation models
- Pretraining paradigms
- DINOv2 (self-supervised ViT)
- Stable Diffusion (text-to-image)

Vaswani et al., “Attention is All You Need”, NeurIPS 2017

A sequence model that replaces recurrence entirely with *self-attention*: every token can directly attend to every other token.

Why it matters

- Parallelizable across the sequence (unlike RNNs).
- Global receptive field from layer 1 onward.
- Scales to billions of parameters.
- Unified architecture for text, images, audio, and more.



Each output embedding aggregates *all* input tokens, weighted by attention.

Why attention? The polysemy problem

Context-dependent meaning

The word *bank* has different meanings:

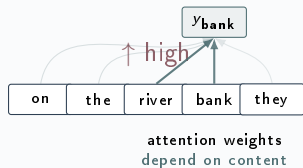
- “on the river **bank**” → riverbank
- “at the **bank**” → financial institution

A static embedding assigns a single vector to “bank” and it cannot capture this ambiguity.

Need for contextual embeddings

A *contextual* representation y_i for token i should depend on the surrounding tokens. The weight given to each neighbour must itself depend on content.

Key idea: “bank” in a river context should attend strongly to “river”; in a financial context to “account” or “money”.



Conclusion

The strength of connections *depends on the tokens themselves*, this is the essence of self-attention.

Token embeddings and positional encoding

Token embedding

Each discrete token index t maps to a dense vector via a learned embedding matrix $E \in \mathbb{R}^{|V| \times d}$:

$$x = E[t] \in \mathbb{R}^d.$$

All positions share the same embedding table.

Sinusoidal positional encoding (Vaswani et al.)

$$\text{PE}_{p,2i} = \sin\left(\frac{p}{10000^{2i/d}}\right)$$

$$\text{PE}_{p,2i+1} = \cos\left(\frac{p}{10000^{2i/d}}\right)$$

The final input to the transformer is $\tilde{x}_p = x_p + \text{PE}_p$.

Why sinusoidal?

- Each dimension oscillates at a different frequency.
- Relative offsets PE_{p+k} are a linear function of $\text{PE}_p \Rightarrow$ model can learn to attend by relative position.
- No maximum sequence length at design time.

Learned positional embeddings

Modern models (ViT, BERT, GPT-2) learn one vector per position. Works well when sequences are short enough. DINOv2 uses learned 2D positional embeddings for patches.

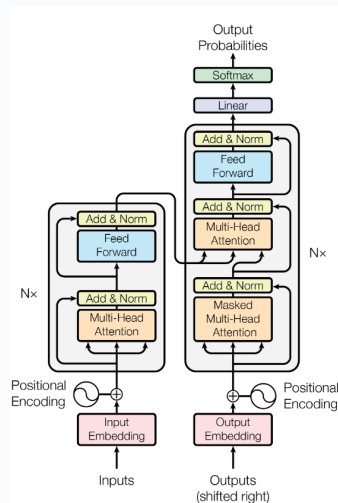
The Transformer architecture (Vaswani et al., 2017)

Encoder

- N identical layers: Multi-Head Self-Attention + Feed-Forward.
- Add & Norm after each sublayer.
- Processes full input in parallel.

Decoder

- Masked self-attention (causal).
- Cross-attention to encoder: Q from decoder, K, V from encoder.
- Auto-regressive at inference.



Vaswani et al. (2017), Fig. 1 "Attention is All You Need"

Scaled dot-product attention

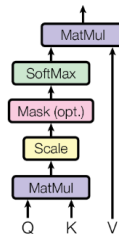
Core formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

$Q, K \in \mathbb{R}^{N \times d_k}, V \in \mathbb{R}^{N \times d_v}$.

- $QK^{\top} \in \mathbb{R}^{N \times N}$: raw compatibility scores.
- $\div \sqrt{d_k}$: prevents softmax saturation for large d_k .
- Softmax row-wise: weights sum to 1 per query.
- Output: weighted average of value vectors.
- **Complexity**: $O(N^2 d)$ time, $O(N^2)$ memory.

Scaled Dot-Product Attention



Vaswani et al. (2017), Fig. 2 (left)

Self-attention: queries, keys, and values

Linear projections from the same input

Q, K, V all come from the *same* input sequence $X \in \mathbb{R}^{N \times d}$:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$W_Q, W_K \in \mathbb{R}^{d \times d_k}, W_V \in \mathbb{R}^{d \times d_v}$ are learned.

Each token projects itself into three roles simultaneously: as a query, a key, and a value.

Intuition: soft dictionary lookup

Think of each token holding a key–value pair, and every other token sending a query:

- **Key** k_j : what information token j offers.
- **Query** q_i : what token i is searching for.
- **Value** v_j : the actual content token j contributes.
- The dot product $q_i^\top k_j$ measures how well i matches j .
- Scaled by $1/\sqrt{d_k}$ to prevent softmax saturation.

Self-attention: computing the output

Attention weights and output

For query i , the attention weight over key j is:

$$\alpha_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{d_k})}{\sum_l \exp(q_i^\top k_l / \sqrt{d_k})}$$

The output for token i is a weighted sum of all values:

$$y_i = \sum_{j=1}^N \alpha_{ij} v_j$$

Matrix form (all tokens in parallel)

$$Y = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- $QK^\top \in \mathbb{R}^{N \times N}$: all pairwise scores at once.
- Softmax row-wise: weights sum to 1 per query.
- $Y \in \mathbb{R}^{N \times d_v}$: contextual output per token.
- Complexity: $O(N^2 d)$ time, $O(N^2)$ memory.

Multi-head attention

Definition

Run h independent attention heads in parallel, each in a $d_k = d/h$ subspace:

$$\text{head}_i = \text{Attn}(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)})$$

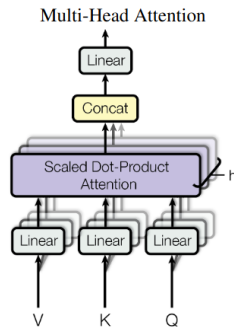
Concatenate and project back:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

where $W_O \in \mathbb{R}^{hd_v \times d}$.

Why multiple heads?

Different heads specialize in different relation types (syntactic, co-reference, positional, semantic). No single head captures all at once; each operates in a different d/h -dimensional subspace.



Vaswani et al. (2017), Fig. 2 (right)

Vision Transformer (ViT) Dosovitskiy et al., ICLR 2021

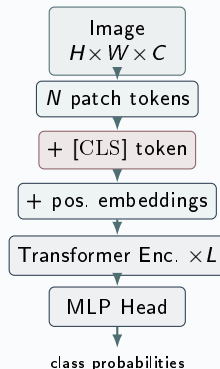
“An Image is Worth 16×16 Words”

Divide the image into N non-overlapping $P \times P$ patches ($N = HW/P^2$). Flatten, linearly project to dimension d , then feed the sequence to a standard transformer encoder.

Pipeline

- 1 Flatten: $f_p \in \mathbb{R}^{P^2 C}$
- 2 Embed: $z_p = W_E f_p + e_p^{\text{pos}}$
- 3 Prepend learnable [CLS] token
- 4 L transformer encoder blocks
- 5 [CLS] output $\rightarrow$ MLP head $\rightarrow$ class probs.

At scale (JFT-303M): ViT-H reaches 88.5% top-1 on ImageNet, outperforming CNNs.



Dosovitskiy et al. (2021)

ViT: patch embeddings

From pixels to tokens

Divide image $H \times W \times C$ into $N = HW/P^2$ non-overlapping $P \times P$ patches. Flatten each patch to a vector $\mathbf{f}_p \in \mathbb{R}^{P^2 C}$ and project with $W_E \in \mathbb{R}^{d \times P^2 C}$:

$$\mathbf{z}_p = W_E \mathbf{f}_p \in \mathbb{R}^d$$

For ViT-B/16: $P=16$, $d=768$, giving $N=196$ tokens from a 224^2 image.

Patch embedding $\equiv$ strided convolution

Applying W_E to all patches is equivalent to a conv layer with kernel size P , stride P , and d output channels, a single non-overlapping convolution pass. After training, the rows of W_E resemble Gabor-like frequency and color detectors, similar to CNN first-layer filters.

Prepend the [CLS] token

A learnable token $\mathbf{z}_0 \in \mathbb{R}^d$ is prepended to the sequence:

$$[\mathbf{z}_0, \mathbf{z}_1, \dots, \mathbf{z}_N] \in \mathbb{R}^{(N+1) \times d}$$

After all transformer layers, $\mathbf{z}_0$'s output aggregates global image information and is used by the MLP head for classification.

Why patches instead of pixels?

A 224×224 image has 50 176 pixels, too many for $O(N^2)$ attention. 16×16 patches reduce N to 196, making self-attention computationally feasible.

ViT: positional encodings

Why positional encodings?

Self-attention is *permutation-invariant*: the output for each token is the same regardless of where it appears in the sequence. Without positional information, the model cannot distinguish “patch in the top-left” from “patch in the bottom-right”.

Learned 1-D positional embeddings

Each of the $N+1$ positions (including [CLS]) gets a dedicated learned vector $e_p^{\text{pos}} \in \mathbb{R}^d$. The final input is:

$$\tilde{z}_p = z_p + e_p^{\text{pos}}$$

All e_p^{pos} are jointly optimized during training.

What the model learns

Visualizing the cosine similarity between positional embeddings reveals that:

- **Nearby patches** develop high cosine similarity in their positional embeddings.
- **Same row / same column** patches cluster together.
- The model recovers *2-D spatial structure* from 1-D learned vectors without any explicit 2-D encoding.

This shows ViT can learn geometry purely from data.

ViT: attention distance and global context

Mean attention distance

For each attention head, compute the average distance (in pixels) between a query patch and the patches it attends to, weighted by the attention weights. Plot this vs. network depth.

Findings from the ViT paper

- **Layer 1:** some heads already attend globally (large distance), while others attend locally much more diverse than CNNs.
- **Deep layers:** most heads attend globally; the model integrates information across the entire image.
- **Key difference from CNN:** CNNs grow their receptive field gradually layer by layer. ViT can “jump” to any patch from the very first layer.

Why this matters

- Global attention at layer 1 allows ViT to capture long-range dependencies that CNNs only develop deep in the network.
- This makes ViT particularly strong at tasks requiring holistic scene understanding.
- Combined with scale, this global view is the key reason ViT-based models like DINOv2 and CLIP outperform CNNs on many benchmarks.

ViT vs. CNN: inductive biases

CNN: strong built-in priors

- **Translation equivariance:** a filter response is the same regardless of position.
- **Locality:** each neuron sees only a fixed neighborhood.
- **Hierarchy:** receptive fields grow layer by layer.

These priors match natural image statistics well $\Rightarrow$ good generalization with limited data.

ViT: minimal built-in priors

- Only assumption: the image is a sequence of patches.
- No locality, no translation equivariance by architecture.
- All spatial structure must be *learned from data*.

Practical implication

- ViT needs large labeled datasets (JFT-300M) to match ResNet on ImageNet.
- With enough data it learns *better* representations than CNNs.
- Self-supervised pretraining (DINO, MAE) partially restores the gap by learning priors from unlabeled data.

Key insight

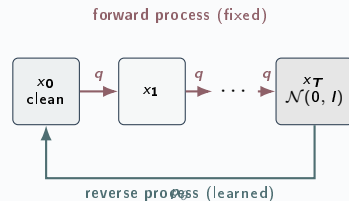
Learned inductive bias (from data at scale) can surpass *hardcoded inductive bias* (from architecture). This is why ViT-based foundation models have displaced CNNs.

Ho et al., “Denoising Diffusion Probabilistic Models”, NeurIPS 2020

Define a *forward Markov chain* that gradually corrupts data with Gaussian noise over T steps, then train a neural network to *reverse* this chain step by step.

Why diffusion?

- **vs. GAN**: no adversarial training, no mode collapse; stable maximum-likelihood training.
- **vs. VAE**: no approximate posterior; the forward process is fixed (not learned).
- State of the art on image, audio, video, and 3-D generation.



Forward process: gradually adding noise

Markov chain

Starting from data x_0 , corrupt it step by step:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

where $\beta_1 < \dots < \beta_T$ is a fixed *noise schedule* (e.g. linearly from 10^{-4} to 0.02).

Closed-form marginal (key result)

Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. Then:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

We can *directly sample any* x_t from x_0 without running the full chain.

What happens as $T \rightarrow \infty$?

$\bar{\alpha}_T \rightarrow 0$, so $q(x_T|x_0) \rightarrow \mathcal{N}(0, I)$ regardless of x_0 .

At $t = T$ all structure is destroyed; we have pure Gaussian noise. Generation starts from here.

Noise schedule choices

- **Linear** (DDPM, Ho et al.): $\beta_t = \text{linear}$, $T = 1000$.
- **Cosine** (Nichol & Dhariwal):
 $\bar{\alpha}_t = \cos^2(\frac{t/T+s}{1+s} \frac{\pi}{2})$. Better quality at $t \approx 0$ and $t \approx T$.
- **Sigmoid, EDM**: modern schedules optimize for fast sampling.

Reverse process and the denoising objective

Reverse Markov chain

The true posterior $q(x_{t-1}|x_t)$ is intractable. We approximate it with a learned Gaussian:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$$

where $\sigma_t^2 = \beta_t$ (fixed) or learned.

Noise-prediction parameterization (Ho et al.)

Instead of predicting μ directly, predict the noise ε :

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right)$$

This is the *epsilon-prediction* parameterization, which works better in practice.

Connection to score matching

Predicting $\varepsilon_\theta \approx \varepsilon$ is equivalent to learning the score function $\nabla_{x_t} \log q(x_t|x_0)$ at each noise level connecting DDPM to score-based generative models (Song & Ermon, 2019).

Simplified loss (Ho et al.)

Drop ELBO reweighting; sample a random t , noise ε , form x_t :

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon} [\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2]$$

DDPM: training and sampling algorithms

Training

- 1 Sample $x_0 \sim p_{\text{data}}$
- 2 Sample $t \sim \text{Uniform}(1, \dots, T)$
- 3 Sample $\varepsilon \sim \mathcal{N}(0, I)$
- 4 Form $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$
- 5 Compute $\ell = \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$
- 6 Gradient step on ℓ w.r.t. θ
- 7 Repeat

No need to run the full chain. Any x_t can be directly sampled from x_0 (using the closed-form marginal).

Sampling (reverse chain)

- 1 Sample $x_T \sim \mathcal{N}(0, I)$
- 2 For $t = T, T - 1, \dots, 1$:
 - Sample $z \sim \mathcal{N}(0, I)$ if $t > 1$, else $z = 0$
 - $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z$
- 3 Return x_0

Requires $T = 1000$ NFEs (network function evaluations). DDIM (Song et al., 2021) reduces this to ~ 50 deterministically.

U-Net: the denoising backbone

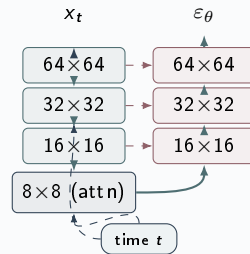
Architecture of $\varepsilon_{\theta}(x_t, t)$

A U-Net adapted for diffusion:

- **Encoder:** ResBlocks + spatial downsampling.
- **Bottleneck:** self-attention over spatial features.
- **Decoder:** upsampling + skip connections.
- **Time cond.:** sinusoidal t embedding added to each ResBlock.

Attention in U-Net

Self-attention at low-res maps (16^2 , 8^2) captures global context efficiently.



Latent Diffusion Models (LDM) Rombach et al., CVPR 2022

The efficiency problem

Running $T = 1000$ U-Net forward passes on 512×512 pixel images is extremely slow. The U-Net must process large spatial tensors at every step.

Key insight: two-stage compression

Stage 1: Train a KL-regularized VAE to encode images:

$$z = \mathcal{E}(x) \in \mathbb{R}^{h \times w \times c}, \quad \hat{x} = \mathcal{D}(z)$$

Typical: $f = 8$ downsampling, so
 $512 \times 512 \times 3 \rightarrow 64 \times 64 \times 4$.

Stage 2: Train a diffusion model entirely *in latent space* $\mathcal{Z}$:

$$\mathcal{L}_{\text{LDM}} = \mathbb{E}_{z, \varepsilon, t} [\|\varepsilon - \varepsilon_\theta(z_t, t)\|^2]$$

The U-Net sees 64×64 tensors $64 \times$ fewer spatial elements than pixel space.

Conditioning via cross-attention

A domain encoder $\tau_\theta(y)$ projects text, segmentation, or class into a shared space. The U-Net attends to it at every layer:

$$\text{Attn}(Q, K, V), \quad K=V=\tau_\theta(y)$$

Why LDM wins in practice

- 64×64 latents: $64 \times$ fewer spatial elements per step.
- VAE compression is near-lossless for perception.
- Decouples generation (LDM) from reconstruction (VAE).
- Basis for Stable Diffusion, DALL-E 3, and most text-to-image models.

Classifier-free guidance and fast samplers

Conditioning on text or class

Train the model with conditioning y (text embedding, class label):

$$\varepsilon_{\theta}(x_t, t, y)$$

During training, randomly drop $y \rightarrow \emptyset$ with probability $p_{\text{uncond}} \approx 0.1$ the model learns *both* conditional and unconditional denoising.

Classifier-free guidance (Ho & Salimans, 2022)

At inference, combine conditional and unconditional predictions:

$$\tilde{\varepsilon} = (1 + w) \varepsilon_{\theta}(x_t, t, y) - w \varepsilon_{\theta}(x_t, t, \emptyset)$$

Guidance scale w : larger $\Rightarrow$ more faithful to prompt, less diverse.

DDIM: deterministic fast sampling

Song et al. (2021) reinterpret the forward process as a non-Markovian ODE:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_{\theta}}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1 - \bar{\alpha}_{t-1}} \varepsilon_{\theta}$$

This allows *subsampling the timesteps*: from $T = 1000$ to ~ 50 steps with similar quality.

Guidance scale tradeoff

- $w = 0$: unconditional, high diversity.
- $w = 7.5$: Stable Diffusion default.
- $w \gg 1$: highly faithful, low diversity, possible artifacts.

Foundation Models

Bommasani et al., “On the Opportunities and Risks of Foundation Models”, 2021

A **foundation model** is trained on broad data at scale, then adapted to many downstream tasks. “Foundation” because these models underpin an entire ecosystem of applications.

Defining properties

- 1 **Scale:** billions of parameters, trillions of tokens.
- 2 **Breadth:** trained on diverse, unlabeled/weakly-labeled data.
- 3 **Emergence:** capabilities appear that were not trained for explicitly.
- 4 **Transferability:** one model adapts to many tasks.

Examples

- **Language:** GPT-4, LLaMA-3, Gemini
- **Vision:** DINOv2, SAM, MAE
- **Multimodal:** CLIP, DALL-E 3, Gemini Vision
- **Audio:** Whisper, AudioLM

Key paradigm shift

Before: design task-specific architectures and train from scratch.

After: train one large model once; fine-tune or prompt for each task. Training cost amortized over thousands of applications.

Pretraining paradigms for vision

Supervised

Large labeled datasets
(ImageNet-21k, JFT-300M).

- Well-understood.
- Limited by annotation cost.
- Labels introduce biases.

Example: ViT (Dosovitskiy et al.)

Self-supervised

Pretext tasks on unlabeled images.

- Masked image modeling (MAE, BEiT).
- Self-distillation (DINO, DINOv2).
- Contrastive (MoCo, SimCLR).

Example: DINOv2 (no labels).

Weakly-supervised / CLIP-style

Align image and text embeddings:

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{N} \sum_i \log \frac{e^{s_{ii}/\tau}}{\sum_j e^{s_{ij}/\tau}}$$

Example: CLIP (400M pairs).
Enables zero-shot transfer.

Adaptation strategies

- **Full fine-tuning:** update all weights on the downstream task.
- **Linear probing:** freeze backbone; train only a linear head on top.
- **Prompt/prefix tuning:** prepend learnable tokens; backbone frozen.
- **LoRA:** add low-rank $\Delta W = AB$ to frozen weight matrices; only A, B trained.

Case Study: DINOv2 Oquab et al., 2023

Goal and contributions

General-purpose visual features *without labels*:

- 1 **Data**: LVD-142M (142M curated images).
- 2 **Recipe**: DINO + iBOT + KoLeo regularization.
- 3 **Scale**: ViT-g/14 at 1.1B parameters.

Results

- ImageNet linear probe: **86.5%** top-1.
- Outperforms CLIP on dense tasks (depth, segmentation).
- Emergent: object segmentation, depth, correspondence.

Why DINOv2 matters

- **No labels needed**: self-supervised training on curated internet images.
- **Universal features**: a single frozen backbone works for classification, segmentation, depth, and retrieval.
- **Data quality over quantity**: curated LVD-142M outperforms random scraping of larger datasets.
- **vs. CLIP**: DINOv2 uses no text — all structure comes from image-only self-distillation.

Case Study: Stable Diffusion Rombach et al., CVPR 2022

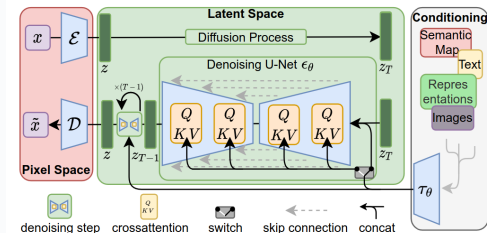
Three-component architecture

- 1 **KL-VAE**: compress $512^2 \times 3 \rightarrow 64^2 \times 4$ (factor $f = 8$).
- 2 **LDM U-Net**: denoise in latent space $\mathcal{Z}$, conditioned on text.
- 3 **CLIP text encoder**: open-vocabulary prompt $\rightarrow$ token sequence τ .

Trained on LAION-5B (5B image-caption pairs).

Division of labour

- VAE $\rightarrow$ perceptual quality (edges, textures).
- LDM $\rightarrow$ semantic content (layout, identity, style).
- CLIP $\rightarrow$ rich open-vocabulary text grounding.



Rombach et al. (2022), Fig. 3

Summary: three pillars and two case studies

Transformers

- Scaled dot-product attention.
- QKV projections from same input.
- Multi-head: multiple relation types in parallel.
- ViT: images as patch sequences.
- Minimal inductive bias; scales with data.

Diffusion Models

- Forward: add noise via Markov chain.
- Reverse: U-Net predicts ϵ .
- Simple MSE loss on noise prediction.
- LDM: compress to latent first.
- CFG: fidelity-diversity tradeoff.

Case Studies

- **DINOv2**: ViT + self-distillation + curated data $\Rightarrow$ universal vision features without labels.
- **Stable Diffusion**: VAE + LDM + CLIP $\Rightarrow$ text-to-image synthesis at scale.

The unifying idea

Scale (data, parameters, compute) combined with the **right architecture** (transformer self-attention) and **right objective** (predict noise / predict masked tokens / align modalities) yields representations general enough to transfer anywhere with or without task-specific supervision.